Adapting Categorical CRDT Lifecycle to Op-Based CRDTs via Replica-Identifier Partitioning

Merkle, B.[1] Rodionov, V.[2]

Draft — preprint, April 2026

Abstract

Yu et al.[1] introduced a lifecycle-counter technique for state-based CRDTs, encoding per-operation state (undone vs. active) in a second timestamp dimension alongside the primary clock. The broader insight in that work is that a CRDT's resolution algebra admits categorical load on a second dimension without breaking convergence. Op-based CRDTs (YATA[2], RGA[3]) lack an explicit second dimension in their standard formulation but expose a structurally similar feature — the replica-identifier tiebreaker — that established practice (notably Jahns' Yjs guidance[4]) treats as opaque. We show that partitioning the replica-identifier space into priority-ordered categorical ranges ports Yu's lifecycle-dimension semantics to op-based CRDTs without protocol changes. The construction also delivers a property Yu's per-operation carrier cannot: because the partition is a uniform order-preserving shift applied to each peer's identifier, the inter-peer tiebreaker is invariant across lifecycle stages — a peer that wins ties in one stage wins in every stage, which is what makes multi-peer collaboration under categorical priorities well-behaved. The whole construction lives inside Lamport's permissive "any total order"[5] formulation, so convergence follows from the base algorithm's correctness. Deployed in Plexus/Yjs, where it underpins deterministic genesis convergence[6], ephemeral-session isolation[7], and lifecycle-aware undo filtering (developed in companion work).

1 Introduction

The core insight — that a CRDT's resolution algebra admits categorical load on a second dimension without breaking convergence — is Yu et al.'s[1], developed in the state-based setting. This paper adapts the insight to op-based CRDTs (YATA, RGA and their families) via a specific construction: partitioning the replica-identifier space into priority-ordered ranges. We describe the construction, the conditions under which it is safe, and how the companion papers in this stack use it.

1.1 Yu et al.'s second dimension (state-based)

State-based CRDTs have a well-defined algebra for concurrent operations: join-semilattice merge. They do not, in standard formulations, support undo — Dolan[8] proves algebraic undo of general CRDTs is impossible beyond counters. Yu et al.[1] attach a per-operation **undo-length counter** alongside the primary clock: even values mark an operation as active, odd values as undone. The counter is a second ordering dimension, carrying *categorical lifecycle state* that the primary clock cannot. Primary-clock convergence is untouched; the counter only affects application-level interpretation.

The broader move — a second timestamp dimension carrying application-semantic load — is the abstraction that matters. Yu's specific payload (undo state) is one instance; any categorical property orthogonal to causal ordering could sit on the same axis.

1.2 The op-based translation problem

Op-based CRDTs have a different shape. They don't have a single "state" object — they have a stream of operations. Their resolution algorithm uses a lexicographic comparison over (Lamport clock, replica-identifier) where the replica-identifier is conventionally a *uniqueness-only* tiebreaker. Established Yjs guidance is explicit: *"clientIDs are session identifiers, not designed to be used for anything else"*[4]. The formal CRDT literature is silent on identifier assignment policy; the opaqueness norm is an implementation default, not a theorem.

Direct application of Yu's technique — attaching a new per-operation counter — would require extending the op-based protocol with a new field on every operation, paying wire-format and compatibility costs on every existing deployment. The question is whether op-based CRDTs have a second dimension *already available* that could carry the same semantic load without a protocol extension.

[1]Independent researcher, Here.build; x.com/merkle_bonsai; Immunefi top–40 whitehat (ethical hacker); email: merklebonsai@pm.me

[2]Independent researcher, Here.build; email: v@here.build

1.3 The construction

We observe that the replica-identifier tiebreaker is exactly that second dimension. The resolution algorithm already reads it, it is already stable across peers, and it is already causally independent of the clock. Lamport's 1978 formulation[5] specified only "an arbitrary total ordering of the processes" — a permissive formulation the field narrowed over subsequent decades into a convention of randomness.

When the replica-identifier space is partitioned into priority-ordered categorical ranges, this existing axis carries Yu-style lifecycle semantics without a protocol change, a new field, or coordination. The construction is inside Lamport's permission (any total order includes priority-ordered ones); convergence follows from the base algorithm's convergence, because only the identifier each operation receives has changed, not the comparison function that consumes it.

Yu's categorical-second-dimension insight and Lamport's permissiveness pre-date the work; we are not claiming those. The contribution has two parts: an *adaptation* and an *extension*.

The present construction is general. Its invariants (P1–P3, §2.1) and convergence preservation (§2.2) depend only on assumptions stated abstractly (operation-based CRDT with numeric replica-identifier tiebreaker; §2.3 Applicability table) and hold across any runtime meeting them. The companion applications that build on this construction — genesis[6], liminality[7], entity-oriented architecture[9], composite keys[10] — are concretely *deployed* in Plexus/Yjs, and each addresses its portability commitments in its own constraints section and, where appropriate, in a dedicated appendix (notably the architecture paper's Appendix A and the liminality paper's Appendices A–C). The primitive of this paper is general; the applications it anchors are Yjs deployments with explicit portability analyses rather than demonstrated ports to Loro or Automerge.

The adaptation is bringing Yu's lifecycle-dimension semantics to op-based CRDTs where folk guidance had foreclosed the technique. The extension is a consequence of the specific carrier we use. Yu attaches his second dimension as a per-operation counter; we attach ours to the per-peer replica identifier. Because the carrier is per-peer and the partition is a uniform order-preserving shift, our construction enjoys a property Yu's doesn't: the inter-peer tiebreaker is invariant across lifecycle stages (§2.1, P3). Who wins ties in one stage wins ties in every stage. This is what makes multi-peer collaboration under categorical priorities well-behaved — users never get inconsistent winner-relationships depending on which lifecycle range their operations occupy.

The paper describes the construction, its invariants (including P3), collision analysis, and the companion applications that motivated it: deterministic genesis convergence[6], liminal state commit-rewrite[7], entity-oriented document architecture with lifecycle-discriminated identity encoding[9].

2 The Construction

Partition the replica-identifier space into non-overlapping ranges, ordered by priority. Yjs's identifier space is bounded by $2^{53} - 1$ (JavaScript safe integer); we use two leading bits as prefix, giving four equal ranges of 2^{51} each:

Prefix	Range	Prio.	Purpose
`0b00`	$[0, 2^{51})$	low	Regular ops
`0b01`	$[2^{51}, 2^{52})$	low	Ephemeral
`0b10`	$[2^{52}, 3{\cdot}2^{51})$	med	Committed
`0b11`	$[3{\cdot}2^{51}, 2^{53})$	high	Genesis

All clientIds derive from a **single 51-bit random value** X generated once per peer:

$$
\begin{aligned}
X &= \texttt{random51()} \text{ (once per peer)} \\
\text{regular:} &\quad X \\
\text{ephemeral } N\text{:} &\quad X + 2^{51} + N \\
\text{committed } N\text{:} &\quad X + 2^{52} + N \\
\text{genesis:} &\quad 3{\cdot}2^{51} + h(\mathit{type}, \mathit{path}) \bmod 2^{51}
\end{aligned}
$$

One random value, four namespaces. The payload (lower 51 bits) is shared across regular and ephemeral ranges — the prefix bits alone distinguish the namespace. Session counting consumes the lowest bits; the random entropy is in the upper bits.

The scheme generalizes. For identifier width W and k equally sized partition categories (with k a power of 2), range i occupies $[i \cdot 2^{W-\log_2 k}, (i+1) \cdot 2^{W-\log_2 k})$. All subsequent formulas scale. Non-power-of-2 k is admissible via $\lceil \log_2 k \rceil$ prefix bits with the remaining prefix slots reserved; this wastes identifier space but does not break the construction. Wider integer platforms (native 64-bit, 128-bit) can use more prefix bits for finer partitioning, more payload bits for lower collision probability, or both. The 2-prefix/51-payload split is specific to JavaScript's 53-bit safe integer constraint, not to the construction.

2.1 Invariants

P1. Category precedence. For operations $a \in R_i$ and $b \in R_j$ with $i < j$, clientId(a) < clientId(b) by partition construction. Under max-wins tiebreaker comparison, b wins when the resolution algorithm reaches the identifier comparison. (Min-wins inverts the range ordering.) The invariant's force depends on how often the tiebreaker is reached in the host algorithm — §2.3 discusses this per-algorithm.

P2. Session ordering. Monotonic allocation within a range ensures later sessions produce higher identifiers on the same peer. Combined with P1, this gives two-level priority: category first, then recency. Cross-peer ordering within a range is not guaranteed — each peer maintains its own counter.

P3. Inter-peer order preservation. The partition is a *uniform shift* applied to every peer's underlying identifier. A peer's clientId in range R_i is $X + \text{base}_i$, where X is the peer's 51-bit random base (the same X reused across every stage the peer enters). Because the shift is order-preserving, the inter-peer tiebreaker is invariant across lifecycle stages: for any two peers A and B,

$$X_A > X_B \implies X_A + \text{base}_i > X_B + \text{base}_i$$

for every range i. A peer that wins ties at one stage wins at every stage. A peer that loses at one stage loses at every stage. Inter-peer resolution is stable regardless of which lifecycle range the conflict arises in.

Yu's carrier is a per-operation counter, not a per-peer identifier; his second dimension has no notion of "peer A's tie-break rank" to preserve across stages. Our carrier is per-peer (the replica identifier), and the partition scheme is an order-preserving shift by construction, which yields P3 directly. Without P3, multi-peer collaboration under categorical priorities would produce inconsistent winner-relationships depending on which lifecycle stage the conflict arose in.

2.2 Convergence preservation

The construction leaves the CRDT's comparison function unchanged; only the identifier-assignment policy differs. Global identifier uniqueness is preserved — the partitioned random range (2^{51} per namespace) is larger than the default Yjs range (2^{32}), strictly reducing collision probability (§3). Convergence follows from the base algorithm's convergence.

Applications that manipulate identifiers beyond assignment — commit-time rewriting of item coordinates[7], deterministic content-addressed IDs for genesis entities[6] — are developed in companion papers and each establishes its own convergence claim over the primitive defined here.

2.3 Applicability

The construction applies to operation-based CRDTs where replica identifiers serve as a tiebreaker in concurrent conflict resolution:

For algorithms where the tiebreaker is secondary (RGA, Fugue), category precedence manifests only in the subset of concurrent operations where primary ordering does not disambiguate. This is where the technique is useful — primary ordering handles the common case, categorical priority handles the lifecycle-sensitive residual.

3 Collision Analysis

Because committed identifiers are permanent and session IDs are clustered around a single peer's random base, the collision model is *birthday on X values with the effective range reduced by session-block size.* Each peer's single 51-bit random value X determines all its clientIds; sessions occupy $X+1 \ldots X+N$ — consuming $\lceil \log_2(N) \rceil$ bits from the lower end.

Collision probability (10 users × 1K reconnects = 10K random bases, $p < 0.01$ required). The 2^{32} columns are the default Yjs baseline; 2^{51} columns reflect the partitioned-range baseline used in the construction:

Safety threshold: ≥ 33 effective random bits for 10K bases at $p < 1\%$. At 2^{51}, the system supports up to ∼262,144 sessions per reconnection ($51 - 18 = 33$ bits) before crossing 1%. At 2^{32}, even 10 sessions is marginal. The uniform 2^{51} range eliminates the need for asymmetric range sizing.

Genesis range collision (content-addressed hash, 2^{51} range):

Types per document	p at 2^{51}
100	2.2×10^{-12}
1K	2.2×10^{-8}
10K	2.2×10^{-8}
100K (1M projects)	2.2×10^{-6}

Failure mode. A genesis collision means two distinct genesis paths hash to the same clientId. Yjs's StructStore identifies structs by `(clientID, clock)`; a second item with the same `(clientID, clock=0)` is treated as a duplicate of the first. The exact behavior (silent drop vs. integration error) depends on the integration path — remote integration may differ from

Algorithm	Identifier role	Applicable
YATA (Yjs)[2]	Final tiebreaker for same-origin inserts and map entries	**Yes**
RGA[3]	Secondary tiebreaker after Lamport timestamp	**Yes** (only when Lamport ties)
Fugue	Final tiebreaker after structural checks	**Plausible** (unverified; only when structural checks tie)
Automerge[11]	Secondary tiebreaker (string actorIds, requires prefix partitioning)	**With adaptation** (actorIds are hex strings; leading-byte prefix partitioning is direct)
Logoot / LSEQ	Embedded in position-path construction	**No** — changes spatial distribution
Woot / TreeDoc	Structural disambiguation, not identifier comparison	**No**
State-based CRDTs	Merge via lattice join, identifiers not compared	**No**

Sessions/base	Bits consumed	Effective bits at 2^{32}	p at 2^{32}	Effective bits at 2^{51}	p at 2^{51}
1	0	32	1.2×10^{-2}	51	$\mathbf{2.2 \times 10^{-8}}$
10	4	28	1.2×10^{-1}	47	$\mathbf{2.2 \times 10^{-7}}$
100	7	25	$\sim$1.0	44	$\mathbf{2.2 \times 10^{-6}}$
1000	10	22	$\sim$1.0	41	$\mathbf{2.2 \times 10^{-5}}$
10000	14	18	$\sim$1.0	38	$\mathbf{2.2 \times 10^{-4}}$

local creation. The observable outcome in either case: the application sees at most one genesis entity at the colliding identifier, and the probability is bounded but the failure is silent from the application's perspective. Applications relying on genesis must tolerate this bound or validate genesis-path uniqueness out-of-band.

The collision analysis uses a non-cryptographic hash (Murmur3). Birthday bounds apply to random inputs. For adversarial inputs, targeted collisions are feasible in seconds of CPU — mitigation requires either cryptographic hashing or a validation layer outside the CRDT (§4, C2).

4 Constraints

C1. Cooperative deployment. The construction assumes participants assign identifiers honestly. A malicious peer can self-assign any clientId — including a genesis-range identifier producing conflict-winning items — and the CRDT will integrate it without objection. Enforcement requires a validation layer (server-side relay, authenticated sync) outside the primitive; this is comparable to how CRDTs generally assume authenticated peer identity.

C2. Adversarial determinism. Partitioning introduces an attack surface random-ID CRDTs do not have: an adversary with random IDs can only hope to win tiebreakers by chance, while an adversary under this scheme can guarantee wins on any contested write by selecting the highest available prefix. The opaqueness convention Jahns recommends[4] incidentally limits this attack (a random ID's expected priority outcome is, well, random); the construction trades that incidental limit for explicit priority semantics. Genesis integrity under adversarial inputs additionally requires either cryptographic hashing or out-of-band validation, since genesis-range assignment uses a non-cryptographic hash (Murmur3) vulnerable to targeted collisions (§3).

C3. Single partition scheme per document. All participants must agree on the range boundaries. Heterogeneous schemes produce divergent priority outcomes — convergence breaks. Agreement is a deployment-time invariant; the primitive does not check it.

C4. Wire format. Identifiers above `uint32` require variable-length encoding. Yjs uses lib0 varUint (supports up to 2^{53}). High-range identifiers (prefixes `0b01`–`0b11`) add 3 bytes to the wire encoding vs regular `uint32`. JavaScript bitwise operators truncate to int32 — all high-range arithmetic must use float64 operations (`Math.floor`, `%`), not `|`, `&`, `«`, `»` on the full value.

C5. Information leakage. Monotonic allocation reveals session count and relative ordering to peers who receive operations from that range. Acceptable for collaborative editing; potentially unacceptable for privacy-sensitive deployments.

5 Applications

The primitive is used in Plexus for techniques developed in companion work:

- **Deterministic genesis**[6] — highest-priority range with content-addressed identifiers for structural entities.
- **Liminal state**[7] — ephemeral and committed ranges for deferred-persistence operation lifecycles.
- **Entity-oriented CRDT documents**[9] — flat-shell document model using the prefix bits for lifecycle discrimination, with the concrete self-resolving UUID encoding and priority-ordered resolution routing operations through structural liveness tiers.
- **Composite entity-addressed keys**[10] — extends decodable entity references into map-key positions with a structured serialization algebra.

Each companion paper establishes its own invariants and convergence claims over the primitive defined here. The primitive itself is intentionally small — a construction plus its invariants — so that downstream papers cite it uniformly without re-establishing its properties.

6 Related Work

Yu et al.[1] is the direct intellectual source. Their per-operation undo-length counter adds a categorical second dimension to state-based CRDT operations — active vs. undone — without breaking convergence. We port that second-dimension pattern to op-based CRDTs using the replica-identifier tiebreaker as the carrier rather than a new counter field. The payload is different (general lifecycle priority rather than undo state specifically), but the structural move — *categorical semantics on a second axis, leaving the primary clock untouched* — is the same.

Lamport[5] specified the identifier-as-tiebreaker pattern as "any arbitrary total ordering of the processes." The construction here fits inside that specification; we are not extending Lamport's formulation but exercising its permissive clause, which the field had narrowed by convention.

Burckhardt[12] formalizes CRDT resolution as two axes — *visibility* (causal) and *arbitration* (total). Burckhardt names the arbitration axis but leaves it unstructured beyond total-order; our technique gives that axis categorical structure without extending the framework. **Kulkarni et al.**[13] (hybrid logical clocks) and **Preguiça et al.**[14] (dotted version vectors) are adjacent precedents for multi-dimensional timestamps — HLC attaches physical time; dot stores structure the identifier for causality tracking rather than arbitration priority. Different payloads; the pattern of "structure a second axis, leave the primary clock untouched" is shared across the lineage.

Shapiro et al.[15] formalize CRDTs using join-semilattices where replica identifiers participate in the merge function. **Weidner et al.**[16] compose CRDTs via semidirect products with an explicit arbitration order, demonstrating that the ordering used for conflict resolution can be semantically meaningful.

Established framework guidance treats replica identifiers as opaque. Jahns' Yjs community guidance[4] — *"clientIDs are session identifiers, not designed to be used for anything else"* — is the most explicit articulation. The guidance is a reasonable default when resolution semantics are unknown; Yu's work demonstrates that with known semantics, structured assignment is safe and useful. The construction here applies that lesson to op-based CRDTs in ecosystems where the opaqueness norm had been treated as prescriptive.

7 Conclusion

This paper shows how to realize Yu et al.'s categorical-second-dimension technique in op-based CRDTs by partitioning the replica-identifier space into priority-ordered ranges. Yu established the semantic move in state-based CRDTs and Lamport's 1978 formulation already permitted structured replica orderings; the contribution here is the construction that makes Yu's categorical second dimension accessible in op-based systems without a protocol extension.

The applications that motivated the adaptation — deterministic genesis convergence[6], liminal state transitions[7], lifecycle-aware identity encoding[9] — each exercise the second-axis semantics for a different purpose. Together they illustrate that the same pattern Yu used for undo-state carries well beyond undo: genesis priority, ephemeral-session isolation, persistence filtering, and UUID-prefix routing are all instances of "categorical load on a second axis" once the axis becomes available.

Other CRDT metadata conventionally treated as opaque — logical-clock bits, operation identifiers, causal dependencies — may admit similar refinement, though that is outside the scope of this paper.

Convergence preservation under the construction is argued at the invariant level (§2.2): unchanged comparison function, preserved identifier uniqueness. Formal verification — e.g. via model-checking a YATA-plus-partitioning specification under concurrent liminal commits, genesis materialization, and partition-delayed sync, or via property-based testing against the Yjs and Loro reference implementations — is future work. The invariants P1–P3 are stated in a form amenable to such mechanization.

References

[1] Weihai Yu, Victorien Elvinger, and Claudia-Lavinia Ignat. A generic undo support for state-based CRDTs. In *Proc. OPODIS*, 2019.

[2] Petru Nicolaescu, Kevin Jahns, Michael Derntl, and Ralf Klamma. Near real-time peer-to-peer shared editing on extensible data types. In *Proc. ACM GROUP*, 2016.

[3] Hyun-Gul Roh, Myeongjae Jeon, Jin-Soo Kim, and Joonwon Lee. Replicated abstract data types: Building blocks for collaborative applications. *Journal of Parallel and Distributed Computing*, 2011.

[4] Kevin Jahns. Globally unique client id. Yjs Community discussion, https://discuss.yjs.dev/t/312, 2020.

[5] Leslie Lamport. Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, 21(7), 1978.

[6] Plexus Project. Deterministic genesis: Coordination-free structural initialization for operation-based CRDTs. Preprint, 2026.

[7] Plexus Project. Liminal state: Deferred-persistence lifecycles for operation-based CRDTs. Preprint, 2026.

[8] Stephen Dolan. The only undoable CRDTs are counters. *arXiv:2006.10494*, 2020.

[9] Plexus Project. Entity-oriented CRDT documents: Architecture, identity, and structural liveness. Preprint, 2026.

[10] Plexus Project. Composite entity-addressed keys for CRDT maps. Preprint, 2026.

[11] Martin Kleppmann and Alastair R. Beresford. A conflict-free replicated JSON datatype. *IEEE TPDS*, 2017.

[12] Sebastian Burckhardt. *Principles of Eventual Consistency*. Foundations and Trends in Programming Languages, 2014.

[13] Sandeep S. Kulkarni, Murat Demirbas, Deepak Madappa, Bharadwaj Avva, and Marcelo Leone. Logical physical clocks. In *Proc. OPODIS*, 2014.

[14] Nuno Preguiça, Carlos Baquero, Paulo Sérgio Almeida, Victor Fonte, and Ricardo Gonçalves. Dotted version vectors: Logical clocks for optimistic replication. *arXiv:1011.5808*, 2010.

[15] Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. A comprehensive study of convergent and commutative replicated data types. *INRIA RR-7506*, 2011.

[16] Matthew Weidner, Heather Miller, and Christopher Meiklejohn. Composing and decomposing op-based CRDTs with semidirect products. *ICFP / PACMPL*, 2020.